

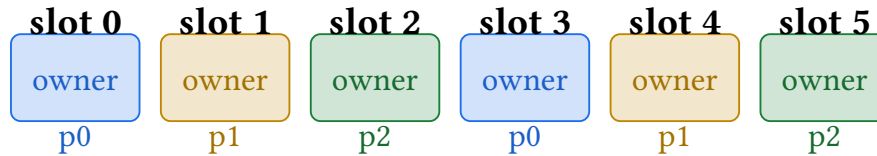
A liveness bug in Mencius Protocol P

How one crashed revoker can wedge the whole log

Mencius (Mao, Junqueira, Marzullo, OSDI 2008) – Protocol P, Appendices A + B

Refresher 1/4 – the replicated log

Mencius replicates a **log**: an array of slots, each decided by one independent consensus instance. Slots are pre-assigned round-robin, $\text{owner}(i) = i \bmod n$ (here $n = 3$):

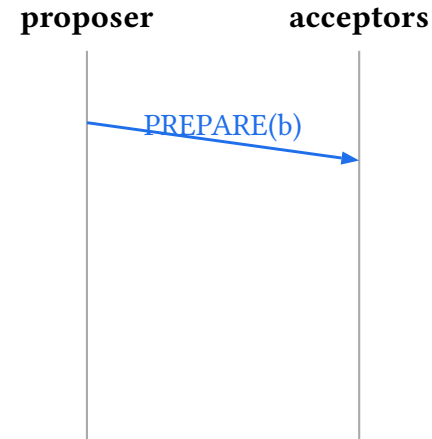


- Commit is **strictly in log order**: slot i is applied only after every slot $< i$ is learned. One un-learned slot blocks everything behind it.
- Today's focus: **slot 2**, owned by **p2**.

Refresher 2/4 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b, ab, av)	Phase 1b: promise + last accept
PROPOSE(b, v)	Phase 2a: accept v at b
ACCEPT(b, v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

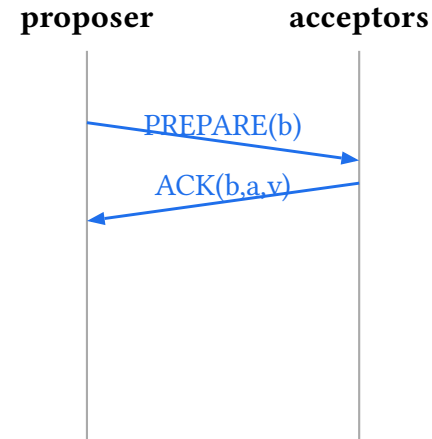


A value is **chosen** once a quorum accepts it at one ballot; thereafter it cannot change.

Refresher 2/4 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b, a, v)	Phase 1b: promise + last accept
PROPOSE(b, v)	Phase 2a: accept v at b
ACCEPT(b, v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

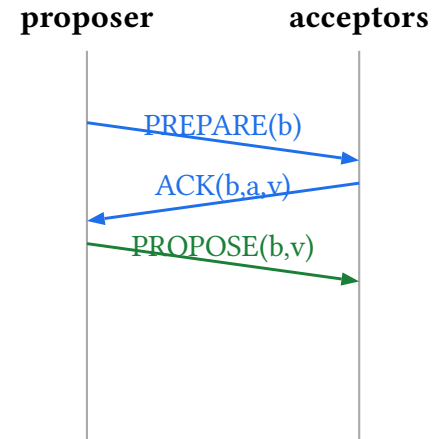


A value is **chosen** once a quorum accepts it at one ballot; thereafter it cannot change.

Refresher 2/4 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b,a,v)	Phase 1b: promise + last accept
PROPOSE(b,v)	Phase 2a: accept v at b
ACCEPT(b,v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

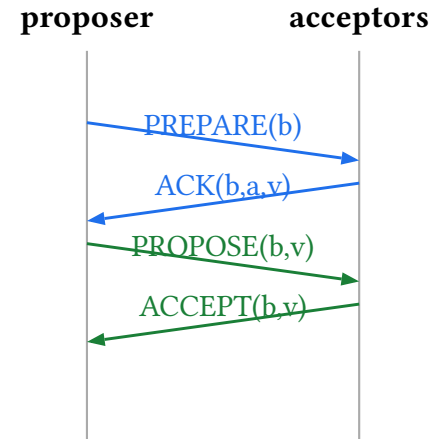


A value is **chosen** once a quorum accepts it at one ballot; thereafter it cannot change.

Refresher 2/4 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b,a,v)	Phase 1b: promise + last accept
PROPOSE(b,v)	Phase 2a: accept v at b
ACCEPT(b,v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen

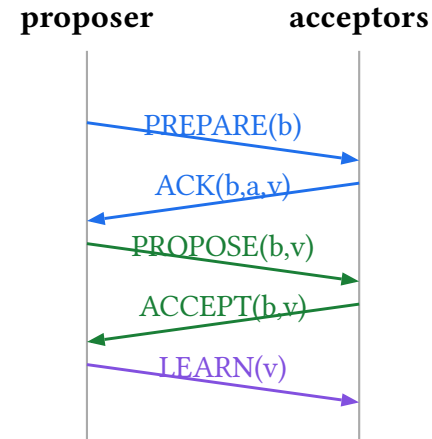


A value is **chosen** once a quorum accepts it at one ballot; thereafter it cannot change.

Refresher 2/4 – Coordinated Paxos messages

Each slot runs one Paxos instance over two phases.

message	role
PREPARE(b)	Phase 1a: ask to lead ballot b
ACK(b,a,v)	Phase 1b: promise + last accept
PROPOSE(b,v)	Phase 2a: accept v at b
ACCEPT(b,v)	Phase 2b: accepted (b, v)
LEARN(v)	quorum accepted: v is chosen



A value is **chosen** once a quorum accepts it at one ballot; thereafter it cannot change.

Refresher 3/4 – two ways a slot gets a value

Fast path (the owner). Every acceptor starts each slot already promised at ballot 0, and the slot's owner is the implicit leader of ballot 0. So the owner skips Phase 1 and just sends $\text{PROPOSE}(0, v)$ – a **SUGGEST** ($\text{PROPOSE}(0, \text{no-op})$ is a **SKIP**).

Refresher 3/4 – two ways a slot gets a value

Fast path (the owner). Every acceptor starts each slot already promised at ballot 0, and the slot's owner is the implicit leader of ballot 0. So the owner skips Phase 1 and just sends $\text{PROPOSE}(0, v)$ – a **SUGGEST** ($\text{PROPOSE}(0, \text{no-op})$ is a **SKIP**).

Revoke (anyone else). If a server **suspects** the owner failed, it takes the slot over: a full Phase 1 + Phase 2 at a **strictly higher** ballot.

Refresher 3/4 – two ways a slot gets a value

Fast path (the owner). Every acceptor starts each slot already promised at ballot 0, and the slot's owner is the implicit leader of ballot 0. So the owner skips Phase 1 and just sends $\text{PROPOSE}(0, v)$ – a **SUGGEST** ($\text{PROPOSE}(0, \text{no-op})$ is a **SKIP**).

Revoke (anyone else). If a server **suspects** the owner failed, it takes the slot over: a full Phase 1 + Phase 2 at a **strictly higher** ballot.

Key asymmetry. The owner only ever uses ballot 0 and **never raises its own ballot**. The only path to a higher ballot is someone else's suspicion-driven Revoke.

Refresher 4/4 – the two safety facts

1. An acceptor accepts $\text{PROPOSE}(b, v)$ **only if** it has not promised a higher ballot and not already accepted at $\geq b$:

$$\text{prepared_ballot} \leq b \text{ and } \text{accepted_ballot} < b$$

Refresher 4/4 – the two safety facts

1. An acceptor accepts $\text{PROPOSE}(b, v)$ **only if** it has not promised a higher ballot and not already accepted at $\geq b$:

$$\text{prepared_ballot} \leq b \text{ and } \text{accepted_ballot} < b$$

2. A value is **chosen** exactly when a **quorum** has accepted it at the same ballot. With $n = 3$, a quorum is 2.

Refresher 4/4 – the two safety facts

1. An acceptor accepts $\text{PROPOSE}(b, v)$ **only if** it has not promised a higher ballot and not already accepted at $\geq b$:

$$\text{prepared_ballot} \leq b \text{ and } \text{accepted_ballot} < b$$

2. A value is **chosen** exactly when a **quorum** has accepted it at the same ballot. With $n = 3$, a quorum is 2.

Consequence we will use: if a revoker raises `prepared_ballot` to 1 on a server, that server will **reject** the owner's later ballot-0 SUGGEST – silently, since Coordinated Paxos has no NACK.

The setup

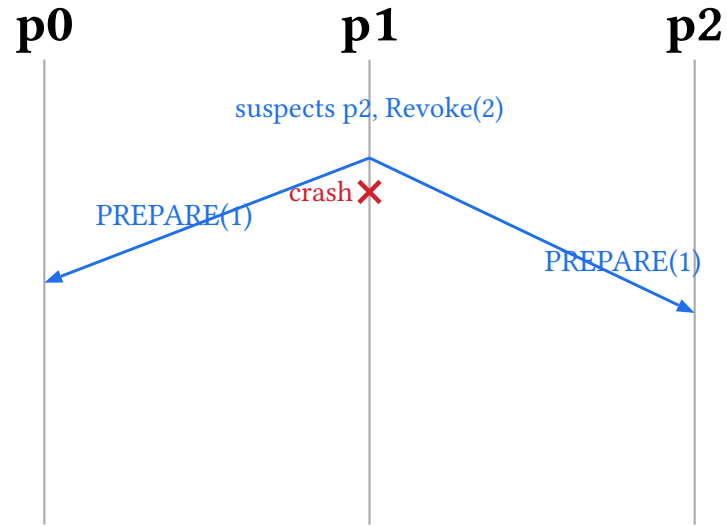
- Three servers **p0, p1, p2**; quorum = 2.
- Slot 2 is owned by **p2**.
- Slots 0–1 already committed everywhere, so all sit at `expected = 2`.
- **p0 and p2 are correct and stay alive.**
- **p1 is faulty – it will crash.**

per-slot state
{prepared_ballot, accepted_ballot,
accepted_value, learned}

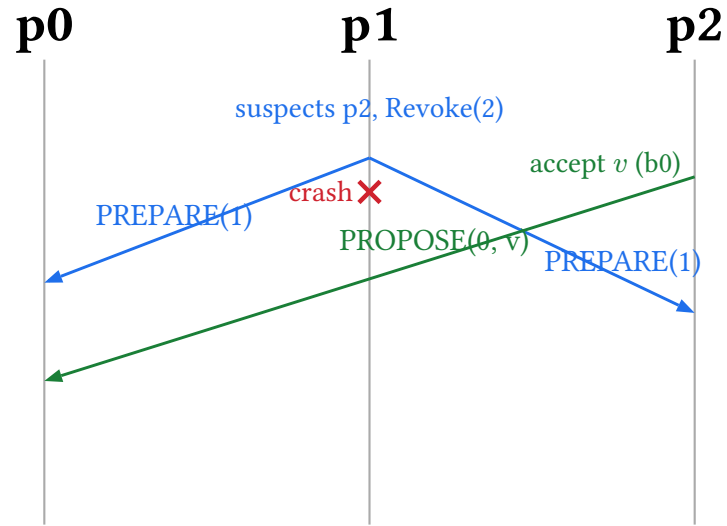
starts at $\{0, -1, \perp, \perp\}$

Ballot 0 is the owner's reserved ballot; a revoke uses any strictly higher ballot the revoker owns.

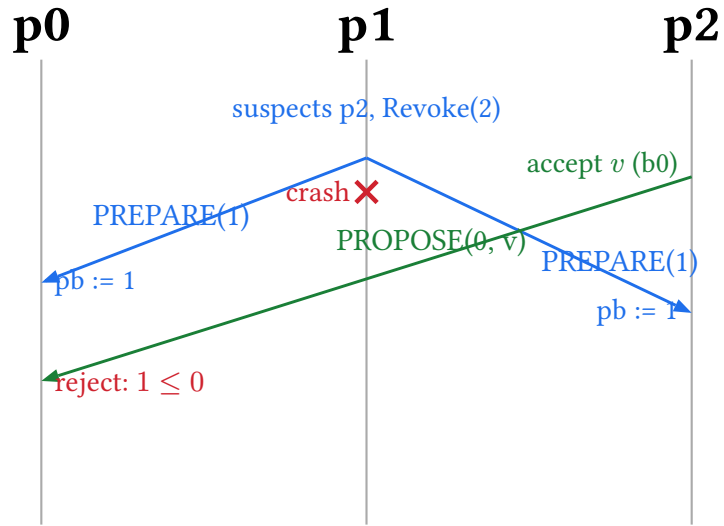
The trace – a crashed revoker orphans slot 2



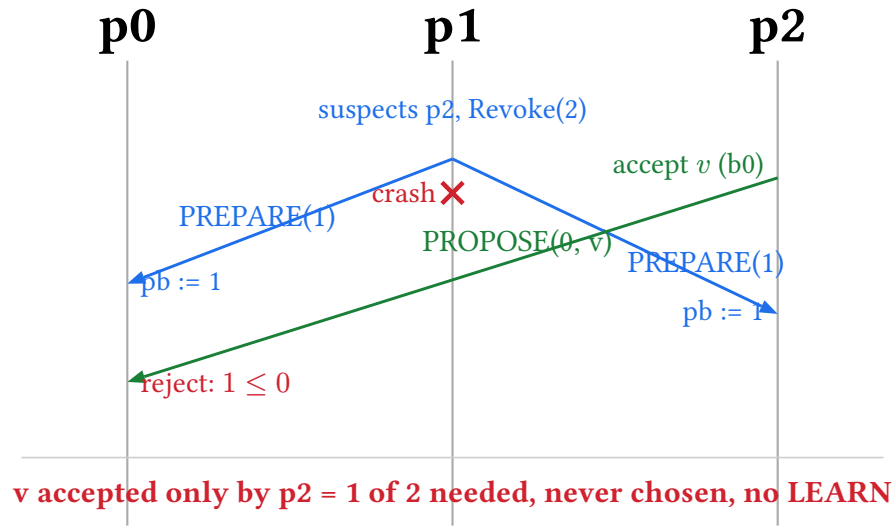
The trace – a crashed revoker orphans slot 2



The trace – a crashed revoker orphans slot 2



The trace – a crashed revoker orphans slot 2



Counterfactual: had **PROPOSE(0, v)** reached **p0** **before** **PREPARE(1)**, **p0** would accept, **p0** and **p2** form a quorum, **v** is chosen. The bug needs **PREPARE-before-SUGGEST** at **p0**.

After the dust settles

slot 2	pb	ab	av	learned
p0	1	-1	$\perp$	$\perp$
p1	crashed			
p2	1	0	v	$\perp$

- v is accepted by **one** server (p2) – below quorum, so **never chosen**.
- No $\text{LEARN}(v)$ is ever broadcast.
- p0 and p2 are both promised at ballot 1, so the owner's ballot-0 fast path is **dead** for slot 2.

Who could ever resolve slot 2?

Resolution needs either (a) a value chosen, or (b) a completing Revoke at a ballot > 1 .

Who could ever resolve slot 2?

Resolution needs either (a) a value chosen, or (b) a completing Revoke at a ballot > 1 .

- **Chosen at ballot 0?** No – p0 rejects further `PROPOSE(0, ...)`, and the owner p2 never escalates above ballot 0.

Who could ever resolve slot 2?

Resolution needs either (a) a value chosen, or (b) a completing Revoke at a ballot > 1 .

- **Chosen at ballot 0?** No – p_0 rejects further $\text{PROPOSE}(0, \dots)$, and the owner p_2 never escalates above ballot 0.
- **p_2 revokes its own slot?** No – OnSuspect only revokes **other** servers' slots; a server never revokes itself.

Who could ever resolve slot 2?

Resolution needs either (a) a value chosen, or (b) a completing Revoke at a ballot > 1 .

- **Chosen at ballot 0?** No – p_0 rejects further $\text{PROPOSE}(0, \dots)$, and the owner p_2 never escalates above ballot 0.
- **p_2 revokes its own slot?** No – OnSuspect only revokes **other** servers' slots; a server never revokes itself.
- **p_0 revokes slot 2?** Only if it suspects p_2 . But p_2 is **correct**, so an eventually-accurate detector ($\diamond P$) eventually stops suspecting it.

Who could ever resolve slot 2?

Resolution needs either (a) a value chosen, or (b) a completing Revoke at a ballot > 1 .

- **Chosen at ballot 0?** No – p0 rejects further `PROPOSE(0, ...)`, and the owner p2 never escalates above ballot 0.
- **p2 revokes its own slot?** No – `OnSuspect` only revokes **other** servers' slots; a server never revokes itself.
- **p0 revokes slot 2?** Only if it suspects p2. But p2 is **correct**, so an eventually-accurate detector ($\diamond P$) eventually stops suspecting it.
- **The crashed p1?** It never ran Phase 2, and it is dead.

Who could ever resolve slot 2?

Resolution needs either (a) a value chosen, or (b) a completing Revoke at a ballot > 1 .

- **Chosen at ballot 0?** No – p_0 rejects further $\text{PROPOSE}(0, \dots)$, and the owner p_2 never escalates above ballot 0.
- **p_2 revokes its own slot?** No – OnSuspect only revokes **other** servers' slots; a server never revokes itself.
- **p_0 revokes slot 2?** Only if it suspects p_2 . But p_2 is **correct**, so an eventually-accurate detector ($\diamond P$) eventually stops suspecting it.
- **The crashed p_1 ?** It never ran Phase 2, and it is dead.
- **Suspecting the dead p_1 ?** Correct – but $\text{OnSuspect}(p_1)$ revokes **p_1 's** slots (1, 4, 7, ...), never slot 2.

Who could ever resolve slot 2?

Resolution needs either (a) a value chosen, or (b) a completing Revoke at a ballot > 1 .

- **Chosen at ballot 0?** No – p_0 rejects further $\text{PROPOSE}(0, \dots)$, and the owner p_2 never escalates above ballot 0.
- **p_2 revokes its own slot?** No – OnSuspect only revokes **other** servers' slots; a server never revokes itself.
- **p_0 revokes slot 2?** Only if it suspects p_2 . But p_2 is **correct**, so an eventually-accurate detector ($\diamond P$) eventually stops suspecting it.
- **The crashed p_1 ?** It never ran Phase 2, and it is dead.
- **Suspecting the dead p_1 ?** Correct – but $\text{OnSuspect}(p_1)$ revokes **p_1 's** slots (1, 4, 7, ...), never slot 2.

Slot 2 is permanently orphaned – and every later slot wedges behind it.

The owner is pinned to ballot 0 and never escalates; the only escalation is suspicion of the owner.

So a **correct** owner can be locked out by a **crashed** revoker – and recovery would require permanently, falsely suspecting a correct server. Multi-Paxos avoids this because a leader just keeps climbing ballots.

A note on reproducing it

This stall is **not** surfaced by the current Spur simulator:

- `monitor_suspensions` in `Mencius_P.spur` suspects **every** peer on **every** tick, so p0 eventually revokes slot 2 and it self-heals – masking the bug precisely by doing the degenerate thing the bug depends on not happening.
- Porcupine checks linearizability (**safety**), not progress.

Surfacing it would need (out of scope here):

- an **eventually-accurate** failure-detector model (stops suspecting correct nodes), and
- a **no-progress / liveness** check.

Full write-up: `liveness_P.md`.